

DUST and DUSTib Pruning in the `dust` Package

Code and mathematical implementation report

October 2026

Abstract

This report compares the two one-constraint pruning policies called `DUST` and `DUSTib` in the current `dust` package. The comparison is made against the derivation in the `DUST` paper, especially the one-dimensional decision function and its closed-form maximizer. The main conclusion is that the two policies are intended to implement the same mathematical pruning test. They are not different statistical methods. `DUST` is a more optimized implementation of the same maximum, whereas `DUSTib` evaluates the decision-function maximizer more literally. The current `DUSTib` code has boundary and stationary-point branches that can cause under-pruning, and it is not used by the Highway SIMD path. The report gives a minimal correction, a more robust design, and a testing plan for making both approaches exact, effective, and fast.

Contents

1 Scope and sources

The mathematical reference used here is the local `DUST` manuscript `PAPER_DUST/DUST.tex`. The relevant material is the section “`DUST` method with one constraint”, the proof of the closed-form maximum in the appendix, and the appendix tables describing the convex conjugates and their domains. In particular, the paper defines the decision function in Equation `eq:decisionDual1D` and derives its maximizer in the proof of Theorem `prop:dual1D_max_exact`.

The implementation examined in this report is:

- `dust/src/1D_DualMaxPolicies.h`, which contains `DualMax_DUST` and `DualMax_DUSTib`;
- `dust/src/1D_DUST_Impl.h`, which computes the dynamic program and calls the selected pruning policy;
- `dust/src/DUST_1D_HW.cpp`, which provides the Highway SIMD implementation and explicitly delegates `DUSTib` to the scalar engine;
- the model policies `1D_[1--8]_*.h`, which provide $\mathcal{D}^*$, its derivatives, domains, boundary limits, and segment costs.

The paper does not use the name `DUSTib`. That name belongs to an implementation variant inherited from the older `dust` code. It is therefore important not to interpret `DUSTib` as a second theoretical pruning method introduced by the paper.

2 Common dynamic-programming problem

Let y_{st} denote the observations $y_{s+1}, \dots, y_t$, with $0 \leq s < t \leq n$. For a one-parameter natural exponential family, the segment cost has the canonical form

$$c(y_{st}; \theta) = (t - s)A(\theta) - \theta S_{st}, \quad S_{st} = \sum_{i=s+1}^t T(y_i), \quad (1)$$

where A is the log-partition function and T is the sufficient statistic. The dynamic-programming cost is

$$Q_t = \min_{0 \leq s < t} \{Q_s + c(y_{st}) + \beta\}, \quad Q_0 = -\beta, \quad (2)$$

where $c(y_{st}) = \min_{\theta} c(y_{st}; \theta)$ and β is the segment penalty in the convention used by the package.

For each possible last change s , define the function-valued cost

$$q_t^s(\theta) = Q_s + c(y_{st}; \theta) + \beta. \quad (3)$$

The purpose of pruning is to remove an index s when it can never be the last change of an optimal future segmentation.

2.1 The one-constraint pruning problem

Suppose that $r < s$. The one-constraint relaxation used by DUST compares q_t^s with q_t^r :

$$\begin{cases} \min_{\theta \in \Theta} q_t^s(\theta), \\ \text{s.t.} \quad q_t^s(\theta) - q_t^r(\theta) \leq 0. \end{cases} \quad (4)$$

The convex conjugate associated with A is

$$\mathcal{D}^*(x) = x(\nabla A)^{-1}(x) - A((\nabla A)^{-1}(x)). \quad (5)$$

The paper uses the following four scalar quantities, which correspond directly to the variables named a, b, c, d in the C++ code:

$$a = \bar{S}_{st} = \frac{S_{st}}{t - s}, \quad b = \bar{S}_{rs} = \frac{S_{rs}}{s - r}, \quad (6)$$

$$c = \bar{Q}_{st} = \frac{Q_t - Q_s}{t - s}, \quad d = \bar{Q}_{rs} = \frac{Q_s - Q_r}{s - r}. \quad (7)$$

3 The decision function and its maximum

After changing variables from the nonnegative Lagrange multiplier to $x = \mu/(1 - \mu)$, the paper obtains the decision function

$$\mathbb{D}(x) = -\mathcal{D}^*(a + x(a - b)) - (c + x(c - d)), \quad x \in [0, x_{\max}]. \quad (8)$$

The test is safe because a positive value proves that the constrained minimum is above the pruning threshold. Thus the pruning condition is

$$\max_{0 \leq x < x_{\max}} \mathbb{D}(x) > 0. \quad (9)$$

The function is concave because $\mathcal{D}^*$ is convex. If $a \neq b$, its stationary point satisfies

$$0 = -(a - b)(\mathcal{D}^*)'(a + x^*(a - b)) - (c - d), \quad (10)$$

$$R = -\frac{c - d}{a - b}, \quad (11)$$

$$m^* = ((\mathcal{D}^*)')^{-1}(R), \quad (12)$$

$$x^* = \frac{m^* - a}{a - b}. \quad (13)$$

In the C++ policies, `R` is the natural-parameter value in the stationarity equation. The model call `DstarPrimeInv(R)` returns m^* . It is a mistake to compare R directly with a boundary expressed in the mean-parameter scale. The current `DUST` code explicitly avoids that mistake by comparing `DstarPrimeInv(R)` with the mean at the boundary.

The maximizer is selected by the following rules:

1. If $x^* \leq 0$, the maximum on the feasible domain is at $x = 0$. This is exactly the PELT test:

$$\mathbb{D}(0) = -\mathcal{D}^*(a) - c. \quad (14)$$

2. If $0 < x^* < x_{\max}$, the maximum is the interior value $\mathbb{D}(x^*)$.
3. If $x^* \geq x_{\max}$ and $x_{\max}$ is finite, the supremum is the limiting value at the endpoint. If $a > b$, the path moves toward the right boundary of the mean domain; if $a < b$, it moves toward the left boundary.
4. If $x_{\max} = +\infty$, the correct treatment depends on the asymptotic derivative and, in equality cases, on the asymptotic value of the decision function. This case must not be confused with a finite endpoint.

The paper states the essential negative-stationary-point rule explicitly: when $x^* < 0$, the decision function is evaluated at zero and the test is the PELT inequality. There is no special mathematical threshold at $x^* = -1$.

3.1 Model domains and endpoint directions

The model-specific conjugates used in the paper and the code are summarized below. The table is also the reason endpoint handling cannot be hard-coded to the left boundary.

Model	$\mathcal{D}^*(m)$	Mean-domain information
Gaussian	$m^2/2$	$m \in \mathbb{R}$
Exponential	$-\log(m) - 1$	$m > 0$
Poisson	$m(\log m - 1)$	$m > 0$
Geometric	$(m - 1)\log(m - 1) - m \log m$	$m > 1$
Bernoulli/Binomial	$m \log m + (1 - m)\log(1 - m)$	$0 < m < 1$
Negative Binomial	$m \log m - (1 + m)\log(1 + m)$	$m > 0$
Variance	$-\frac{1}{2}(\log m + 1)$	$m > 0$

For Bernoulli and Binomial, the domain is bounded on both sides. Therefore, when $a > b$, $x_{\max}$ is a right-boundary value and the endpoint must use $\mathcal{D}^*(1)$, not $\mathcal{D}^*(0)$. For all other models, finite $x_{\max}$ in the $a < b$ direction reaches the left boundary. The paper's Table `dual.table` gives the corresponding formulas for $x_{\max}$.

4 What DUST and DUSTib mean in the code

4.1 DualMax_DUST

The current `DualMax_DUST` implementation in `1D_DualMaxPolicies.h:76--157` is an optimized closed-form test. Its main steps are:

1. compute a, b, c, d and the model-dependent domain limit;
2. handle a boundary value or a linear decision function;
3. evaluate the derivative at $x = 0$; if the derivative is negative, return the PELT test immediately;
4. handle an infinite domain using the derivative at infinity;
5. for a finite domain, compute the actual critical mean $(\mathcal{D}^*)'^{-1}(R)$ and compare it with the mean at the endpoint;
6. for an interior maximum, use the segment-cost shortcut `Model::costEval(..., R, ...)` instead of explicitly evaluating the full decision function at x^* .

The final shortcut is mathematically equivalent to evaluating $\mathbb{D}(x^*)$. It follows from the Legendre relation and the stationarity equation, and it avoids constructing x^* and the interior mean when the critical point is known to be feasible.

4.2 DualMax_DUSTib

The current `DualMax_DUSTib` implementation in `1D_DualMaxPolicies.h:159--202` takes a more literal route:

1. compute R ;
2. compute $m^* = ((\mathcal{D}^*)')^{-1}(R)$ and $x^* = (m^* - a)/(a - b)$;
3. if the point is considered interior, evaluate

$$-\mathcal{D}^*(a + x^*(a - b)) - (c + x^*(c - d)); \quad (15)$$

4. otherwise use a PELT or boundary/asymptotic branch.

The intended formula is therefore the same as `DUST`. The difference is implementation strategy, not a different pruning criterion:

Property	DUST	DUSTib
Mathematical target	Maximum of Equation (??)	Same maximum
Interior evaluation	Legendre/segment-cost shortcut	Direct decision-function value
Early PELT branch	Checks the derivative at $x = 0$ first	Computes the inverse derivative before its PELT branch
Endpoint handling	Selects left or right boundary from the direction of $a - b$	Current code uses the left boundary in both finite-endpoint branches
Highway path	Hand-vectorized for DUST	Explicitly falls back to the scalar engine
Status in documentation	Default and recommended exact rule	Marked “not yet working” in the R documentation

5 Why DUST is faster

There are four separate reasons. They should not be conflated.

5.1 The main reason: SIMD is unavailable for DUSTib

The file `DUST_1D.HW.cpp:53--62` says that DUSTib is not hand-vectorized and delegates to the scalar dust engine. The dispatch at lines 109–115 confirms that DUSTib calls `run_dust_engine<Model, DualMax_DUSTib<Model>>`. In contrast, DUST uses the Highway implementation when the package is built with Highway. Its active candidates are stored in a flat array so that candidate values can be loaded in SIMD lanes. The scalar engine uses the linked-list style active-index structure in `1D_Indices.h`; this structure is not directly vectorizable.

Consequently, comparing `DUST_1D.HW` with method DUST to the same entry point with method DUSTib is primarily a SIMD-versus-scalar comparison as well as a pruning-rule comparison.

5.2 DUST does less work per surviving candidate

The DUST policy checks the derivative at zero before calculating the inverse derivative. When the derivative is negative, the maximum is already known to be the PELT point, so the expensive inverse and endpoint calculations are skipped. The DUSTib policy currently computes `DstarPrimeInv(R)` and x^* before making its negative-point decision.

For an interior maximum, DUST uses `costEval`, which is a compact model-specific expression for the value at the stationary point. DUSTib constructs the stationary mean and evaluates $\mathcal{D}^*$ plus the affine term. Both are constant-time operations, but the latter generally performs more floating-point work and more domain-sensitive logarithm evaluations.

5.3 Under-pruning increases later dynamic-programming work

Any missed positive decision-function value leaves a candidate in the active set. The next iteration then evaluates that candidate in the OP step, and it can be tested again at later times. Thus a branch error in DUSTib does not change the exact segmentation, provided it only causes under-pruning, but it can substantially increase runtime. The current DUSTib negative-point and endpoint branches can cause precisely this type of under-pruning.

5.4 The comparison must be benchmarked fairly

On a scalar build, or when comparing `dust_1D` rather than `DUST_1D.HW`, the SIMD explanation does not apply. The remaining difference is then due to operation count, branch behavior, active-set size, and numerical branch outcomes. Benchmarks should report separately:

- scalar DUST versus scalar DUSTib;
- Highway DUST versus scalar DUSTib;
- final active-set size and the complete active-set trajectory;
- identical data, penalty, model normalization, compiler, and hardware.

6 DUSTib corrections needed for full effectiveness

6.1 Correction 1: every negative x^* is a PELT case

The current code contains:

```
if(xstar <= 0 && xstar > -1)
{
    return (- Model::Dstar(a) -c > 0);
}
```

The lower bound -1 has no role in the decision-function domain. The change of variables used in the paper gives $x \geq 0$, so any $x^* < 0$ is outside the feasible domain, regardless of how negative it is. The literal correction is:

```
if (xstar <= 0.0)
{
    return (-Model::Dstar(a) - c > 0);
}
```

This is exactly the statement in the paper’s proof: when $x^* < 0$, the maximum is at zero and the test is PELT. The same rule should be used with a small numerical tolerance, for example $x^* \leq \varepsilon_x$.

6.2 Correction 2: use the boundary reached by the path

The current DUSTib code uses `Dstar_leftboundary()` in both finite endpoint branches:

```
return (-Model::Dstar_leftboundary()
        - (c + x_max * (c-d))) > 0;
```

This is wrong for a bounded-right-domain model when $a > b$. Since $m(x) = a + x(a - b)$, the path moves upward when $a > b$ and reaches the right boundary. The endpoint value must therefore be selected as:

```
? Model::Dstar_rightboundary()
: Model::Dstar_leftboundary();

return (-Dstar_at_xmax
        - (c + x_max * (c-d))) > 0;
```

This matters for Bernoulli and Binomial, whose mean domain is $(0, 1)$ and whose right boundary is 1. The current DUST policy already uses this directional selection at lines 146–152; DUSTib should use the same logic.

6.3 Correction 3: do not treat an infinite domain as a finite endpoint

The current test

```
if(xstar >= x_max) // case finite only
```

is not actually restricted to a finite $x_{\max}$. If both values are infinite, the expression can enter a branch containing $x_{\max}(c - d)$, which may be infinite or undefined. The finite endpoint branch should explicitly require:

```
if (std::isfinite(x_max) && xstar >= x_max)
{
    // evaluate the directional boundary limit
}
```

The $x_{\max} = +\infty$ case should instead be handled by a model policy that provides the derivative limit at infinity and, when necessary, the value of the decision function at infinity.

6.4 Correction 4: make out-of-domain stationary points explicit

For Exponential, Geometric, Negative Binomial, and Variance models, the natural parameter has a one-sided domain. The inverse derivative `DstarPrimeInv(R)` is meaningful only when R belongs to the range of `DstarPrime`. If it returns NaN, an invalid mean, or an infinite mean, the code must not silently continue to the generic asymptotic branch.

A robust policy should expose a model-level operation that returns either a valid critical mean or an “out of range” result. An out-of-range critical point is then handled by the appropriate endpoint rule. A safe numerical fallback is to return “do not prune” if the endpoint cannot be evaluated reliably; this sacrifices speed but preserves exactness.

6.5 Correction 5: replace the ambiguous asymptotic helper

The scalar policies expose `Dstar_superLinearLimit()`, but the DUSTib final branch uses it as if it were enough to decide the behavior at an unbounded endpoint. Its name and values do not consistently represent the limit of $\mathcal{D}^*$. For example, for the Exponential model, $\mathcal{D}^*(m) = -\log(m) - 1 \rightarrow -\infty$ as $m \rightarrow \infty$, while $(\mathcal{D}^*)'(m) \rightarrow 0$.

The model interface should distinguish at least:

```
DstarPrimeAtInfinity()    // limit of the derivative
dualAtInfinity(c, d, a, b) // sign/value of the full endpoint limit
```

The first quantity is enough when the asymptotic slope is strictly positive or negative. The second is needed in equality cases, such as an asymptotic slope of zero. The current DUST policy’s use of `DstarPrime(+infinity)` is closer to the correct first-order test, but it should also document and handle equality cases explicitly.

6.6 A corrected branch structure

The following is a schematic replacement for the DUSTib branch logic. It is not a complete patch because the scalar model interface first needs a well-defined invalid-domain and asymptotic result, but it shows the correct ordering:

```
// a, b, c, d already computed; require a != b here.
const double f0 = -Model::Dstar(a) - c;
const double g0 = -(a-b) * Model::DstarPrime(a) - (c-d);
```

```

// The feasible decision-function domain starts at x = 0.
if (g0 <= 0.0) return f0 > 0.0;

const double R = -(c-d) / (a-b);
const CriticalMean critical = Model::criticalMean(R);

if (!critical.valid) {
    // The stationary point is outside the mean domain.
    // Select the endpoint from the sign of the derivative/domain direction.
    return endpointOrAsymptoticTest<Model>(a, b, c, d);
}

const double xstar = (critical.mean - a) / (a-b);
const double xmax = Model::xMax(a, b);

if (xstar <= 0.0) return f0 > 0.0;

if (std::isfinite(xmax) && xstar >= xmax)
    return directionalEndpointTest<Model>(a, b, c, d, xmax);

if (std::isfinite(xstar) && xstar < xmax)
    return decisionAt<Model>(a, b, c, d, xstar) > 0.0;

return asymptoticTest<Model>(a, b, c, d);

```

At minimum, the first two literal corrections should be applied immediately: replace the -1 condition by $x^* \leq 0$ and select the endpoint boundary according to the sign of $a - b$. The remaining changes make the method robust at domain and infinity boundaries rather than merely fixing the common interior cases.

7 Improvements for both approaches

7.1 Use one mathematically authoritative maximization kernel

The two policies currently duplicate delicate model-dependent branch logic. That makes it easy for one policy to receive a boundary fix while the other does not. A better design is to centralize the following operations:

- the domain of the mean statistic;
- the map $(\mathcal{D}^*)'$ and its inverse;
- left and right boundary values;
- the directional endpoint selected by $a - b$;
- finite and infinite endpoint evaluation;
- a checked stationary-point result.

Then DUST and DUSTib can differ only in how they evaluate a valid interior maximum. DUST can retain the segment-cost shortcut; DUSTib can retain the direct decision-function formula for diagnostic purposes. Both must call the same endpoint and invalid-domain helpers.

7.2 Make DUSTib a debug/reference mode, not a second unchecked kernel

Once the shared maximization logic is available, DUSTib is useful as a reference implementation. For every candidate, it can evaluate the direct decision function and compare its result with the DUST shortcut in debug builds. This provides a strong regression check for the Legendre shortcut and for model-specific formulas without imposing the extra work on the production path.

7.3 Vectorize only after correctness is established

If DUSTib is retained as a user-facing method, it should either use the same Highway lane kernel as DUST or be documented as scalar reference code. A second hand-vectorized branch tree would increase maintenance cost. The preferred performance path is to vectorize the shared decision test and use lane masks for the PELT, interior, endpoint, and asymptotic cases.

7.4 Strengthen numerical safeguards

The implementation should use model-specific stable primitives and checked domains throughout:

- use `log1p` and stable logistic evaluations near 0 and 1;
- avoid evaluating $\log(0)$ or $0 \log(0)$, even when a floating-point average is within a few ulps of a boundary;
- use a named tolerance policy rather than scattered constants such as `1e-9` and `1e-14`;
- distinguish a finite endpoint, an infinite endpoint, and an invalid stationary point;
- never prune when the decision value is NaN or the domain status is uncertain.

The last rule is conservative: it may reduce pruning, but it cannot destroy the exactness guarantee.

7.5 Test pruning decisions, not only final segmentations

The current tests in `tests/testthat/test-dust1D.R` verify that the methods return the same final change points on selected examples. That is necessary but insufficient. Different pruning schedules can produce the same final segmentation even when one method misses many valid pruning opportunities.

The test suite should additionally compare each policy with an independent reference maximization of Equation (??) on small examples. The reference should test:

1. x^* slightly below zero and far below zero;
2. an interior x^* ;
3. x^* beyond a finite left endpoint;
4. x^* beyond a finite right endpoint for Bernoulli/Binomial;

5. $x_{\max} = +\infty$;
6. $a = b$, a at a model boundary, and a numerically close to a boundary;
7. invalid or non-finite inverse-derivative values.

The reference should compare the Boolean pruning decision and the active-set trajectory, not merely Q_t and the backtracked change points. For small n , an exhaustive grid plus a high-accuracy one-dimensional optimizer is adequate. The production policy should never prune when the reference says the maximum is non-positive.

7.6 Improve candidate selection for higher-dimensional extensions

The paper recommends the nearest smaller active index as an effective single-constraint choice, which is what the current one-dimensional code uses. This is a good default for one-parameter models. For multi-parameter models, stronger lower bounds can be obtained by using several constraints, up to the parameter dimension under the paper’s strong-duality result. The implementation should therefore separate:

- the candidate-selection policy;
- the number of constraints;
- the maximizer for the resulting one- or multi-dimensional decision function.

This avoids treating a one-dimensional closed form as if it automatically solved the multi-parameter problem.

7.7 Keep benchmarks comparable

The paper’s conclusions concern exact pruning and measured computation time, not just arithmetic complexity. Future comparisons should record the model normalization, penalty convention, active-set size, hardware, compiler, scalar/SIMD backend, and whether the reference method is allowed to apply a smallest-index PELT check. In particular, the current `dust.object.1D.HW` documentation correctly notes that `DUSTib` falls back to the scalar object; this fact must be visible in any speed table.

8 Conclusions

The DUST paper gives one one-constraint decision function and one closed-form maximum for the one-parameter case. The repository names two implementations of that idea `DUST` and `DUSTib`; they should have the same mathematical result. `DUST` is faster because it performs an early PELT check, uses a Legendre/segment-cost shortcut, and, in the Highway entry point, is vectorized. `DUSTib` currently performs more scalar work and falls back to the scalar engine.

The most important `DUSTib` correction is to replace the condition $-1 < x^* \leq 0$ by the mathematically correct condition $x^* \leq 0$. The next essential correction is to use the right boundary when $a > b$, particularly for Bernoulli and Binomial models. Infinite endpoints and out-of-domain stationary points should then be handled through an explicit model-policy interface rather than through the current ambiguous final branch.

After these corrections, the two policies should share a tested maximization kernel. `DUST` can remain the optimized production method, while `DUSTib` can be retained as a transparent reference/debug implementation or vectorized if a separate production option is genuinely needed.

References

- [1] V. Runge, C. Truong, and S. Querné. DUST: A Duality-Based Pruning Method For Exact Multiple Change-Point Detection. Local source: `PAPER_DUST/DUST.tex`, especially the decision-function equation and the proof of the one-dimensional maximum.
- [2] `dust` source code. In particular: `src/1D_DualMaxPolicies.h`, `src/1D_DUST_Impl.h`, and `src/DUST_1D_HW.cpp`.